Name : Amruth Ayaan Gulawani
Roll : 2023BEC0050

Github : https://github.com/NotCleo/CS322-ML/tree/main/Logistic-Regression

All the code sits in the above URL, much easier to read than on this PDF!!

Assignment :

Consider the log-likelihood function for logistic regression:

$$L(\theta) = \sum_{i=1}^{m} y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_\theta(x^{(i)}))$$

For the following, you will need to calculate the value of the Hessian H of the above function.

(a) **(10 points)** The files logisticX.csv and logisticY.csv contain the inputs ($x^{(i)} \in R^2$) and outputs ($y^{(i)} \in \{0, 1\}$) respectively for a binary classification problem, with one training example per row. Implement[1] Newton's method for optimizing $L(\theta)$, and apply it to fit a logistic regression model to the data. Initialize Newton's method with $\theta = \vec{0}$ (the vector of all zeros). What are the coefficients θ resulting from your fit? (Remember to include the intercept term.)

(b) **(5 points)** Plot the training data (your axes should be x_1 and x_2 , corresponding to the two coordinates of the inputs, and you should use a different symbol for each point plotted to indicate whether that example had label 1 or 0). Also plot on the same figure the decision boundary fit by logistic regression. (i.e., this should be a straight line showing the boundary separating the region where $h(x) > 0.5$ from where $h(x) \leq 0.5$.)

Notes:

- You should submit all your code as well as any graphs that you might plot. Do not submit answers to theoretical questions.
- Do not submit the datasets.
- Include a **single write-up (pdf) file** which includes a brief description for each question explaining what you did. Include any observations and/or plots required by the question in this single write-up file.
- You should use Python for all your programming solutions.
- Your code should have appropriate documentation for readability.
- You will be graded based on what you have submitted as well as your ability to explain your code.

Here's how I did the assignment :

- Logistic regression is just linear regression but with the [z] being passed through the sigmoid function and the loss function is different
- SInce the previous assignments were about linear regression, I reused the C code from that and just modified the logic in terms of how the csv files are read and copied into the matrices
- The hessian calculation proof is demonstrated below

See the below reference
https://thelaziestprogrammer.com/sharrington/math-of-machine-learning/solving-logreg-newtons-method

Hessian Calculation of log-likelihood function

We start with the log-likelihood function for Logistic Regression, which is the function we want to maximize:

$$L(\theta) = \sum_{i=1}^{m} \left[y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_\theta(x^{(i)})) \right]$$

Where $h_\theta(x^{(i)})$ is the Sigmoid function:

$$h_\theta(x) = \frac{1}{1 + e^{-\theta^T x}}$$

Before differentiating $L(\theta)$, it is highly useful to establish the derivative of the sigmoid function with respect to a single weight parameter, θ_j. Due to the properties of the exponential function, this resolves neatly:

$$\frac{\partial}{\partial \theta_j} h_\theta(x) = h_\theta(x)(1 - h_\theta(x)) \cdot x_j$$

We take the partial derivative of $L(\theta)$ with respect to one specific weight parameter, θ_j, applying the chain rule:

$$\begin{aligned}
\frac{\partial L}{\partial \theta_j} &= \sum_{i=1}^{m} \left[y^{(i)} \frac{1}{h_\theta(x^{(i)})} \frac{\partial h_\theta}{\partial \theta_j} + (1 - y^{(i)}) \frac{1}{1 - h_\theta(x^{(i)})} \left(-\frac{\partial h_\theta}{\partial \theta_j} \right) \right] \\
&= \sum_{i=1}^{m} \left[\frac{y^{(i)}}{h_\theta(x^{(i)})} - \frac{1 - y^{(i)}}{1 - h_\theta(x^{(i)})} \right] \frac{\partial h_\theta}{\partial \theta_j}
\end{aligned}$$

Next, we find a common denominator for the terms inside the brackets:

$$\begin{aligned}
\frac{\partial L}{\partial \theta_j} &= \sum_{i=1}^{m} \left[\frac{y^{(i)}(1 - h_\theta(x^{(i)})) - (1 - y^{(i)}) h_\theta(x^{(i)})}{h_\theta(x^{(i)})(1 - h_\theta(x^{(i)}))} \right] \frac{\partial h_\theta}{\partial \theta_j} \\
&= \sum_{i=1}^{m} \left[\frac{y^{(i)} - y^{(i)} h_\theta(x^{(i)}) - h_\theta(x^{(i)}) + y^{(i)} h_\theta(x^{(i)})}{h_\theta(x^{(i)})(1 - h_\theta(x^{(i)}))} \right] \frac{\partial h_\theta}{\partial \theta_j} \\
&= \sum_{i=1}^{m} \left[\frac{y^{(i)} - h_\theta(x^{(i)})}{h_\theta(x^{(i)})(1 - h_\theta(x^{(i)}))} \right] \frac{\partial h_\theta}{\partial \theta_j}
\end{aligned}$$

Now, substitute the derivative of the sigmoid function we defined earlier:

$$\frac{\partial L}{\partial \theta_j} = \sum_{i=1}^{m} \left[\frac{y^{(i)} - h_\theta(x^{(i)})}{h_\theta(x^{(i)})(1 - h_\theta(x^{(i)}))} \right] \left[h_\theta(x^{(i)})(1 - h_\theta(x^{(i)})) \cdot x_j^{(i)} \right]$$

The complex denominators cancel out completely, yielding the simplified gradient:

$$\frac{\partial L}{\partial \theta_j} = \sum_{i=1}^{m} (y^{(i)} - h_\theta(x^{(i)})) x_j^{(i)}$$

The Hessian matrix is the matrix of second-order partial derivatives. To find the element at row j and column k (H_{jk}), we differentiate the gradient with respect to θ_k:

$$\begin{aligned}
H_{jk} &= \frac{\partial}{\partial \theta_k} \left(\frac{\partial L}{\partial \theta_j} \right) \\
&= \frac{\partial}{\partial \theta_k} \sum_{i=1}^{m} (y^{(i)} - h_\theta(x^{(i)})) x_j^{(i)}
\end{aligned}$$

Since $y^{(i)}$ and $x_j^{(i)}$ are constants with respect to θ, the derivative only applies to the prediction $h_\theta(x^{(i)})$:

$$H_{jk} = \sum_{i=1}^{m} \left(0 - \frac{\partial h_\theta(x^{(i)})}{\partial \theta_k} \right) x_j^{(i)}$$

We substitute the derivative of the sigmoid function one final time:

$$H_{jk} = \sum_{i=1}^{m} - \left[h_\theta(x^{(i)})(1 - h_\theta(x^{(i)})) \cdot x_k^{(i)} \right] x_j^{(i)}$$

Rearranging the terms yields the final scalar equation for a single element in the Hessian:

$$H_{jk} = -\sum_{i=1}^{m} h_\theta(x^{(i)})(1 - h_\theta(x^{(i)})) x_j^{(i)} x_k^{(i)}$$

For computational efficiency, this is typically expressed in matrix notation:

$$H = -X^T S X$$

Where X is the design matrix and S is a diagonal weight matrix where the elements are:

$$S_{ii} = h_\theta(x^{(i)})(1 - h_\theta(x^{(i)}))$$

Hessian is negative-definite, Because $h(x)$ is a probability between 0 and 1, the term $h(1 - h)$ is strictly positive. This guarantees that the Hessian is negative-definite, proving the log-likelihood function is concave and possesses a single global maximum.

what's negative definite? A negative definite matrix is a symmetric matrix where the quadratic form for all non-zero vectors

So, this assignment is more focussed on the newton raphson approach, which is a lot of matrix math, so here's how the C code works :

1) Our CSV_X is two columns and 100 values and our CSV_Y is one column and 100 values of 0 and 1
2) Copied the values from the two CSV to a matrix and ran the following algorithm

1. X **(The Design Matrix):** Dimension $m \times 3$.

$$X = \begin{bmatrix} 1 & x_{1,1} & x_{1,2} \\ 1 & x_{2,1} & x_{2,2} \\ \vdots & \vdots & \vdots \\ 1 & x_{m,1} & x_{m,2} \end{bmatrix}$$

   (Note the column of 1s you added for the intercept).

2. y **(The Target Vector):** Dimension $m \times 1$.

$$y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}$$

3. θ **(The Weight Vector):** Dimension 3×1. Initialized to zeros.

$$\theta = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

Step 2: The Forward Pass (Calculating Prediction)

Inside your `for(int i = 0; i < m_count; i++)` loop, you pull out a single row of X, let's call it $x^{(i)}$, which has dimensions 3×1.

You calculate the dot product:

$$z^{(i)} = \theta^T x^{(i)} = \begin{bmatrix} \theta_0 & \theta_1 & \theta_2 \end{bmatrix} \begin{bmatrix} 1 \\ x_{i,1} \\ x_{i,2} \end{bmatrix} = \theta_0(1) + \theta_1(x_{i,1}) + \theta_2(x_{i,2})$$

- **Code:** `for(int j = 0; j < d; j++) z += theta[j] * X[i][j];`
- **Result:** z is a scalar (a single number).

Then you pass z through the sigmoid function to get the predicted probability h:

$$h^{(i)} = \frac{1}{1 + e^{-z^{(i)}}}$$

- **Result:** h is a scalar between 0 and 1.

Step 3: Accumulating the Gradient (∇J)

The Gradient tells you which direction to step to reduce the error. For the entire dataset, the math formula is:

$$\nabla J = \frac{1}{m} X^T (\hat{y} - y)$$

- Dimensions: $(3 \times m) \times (m \times 1) \to 3 \times 1$ vector.

Because doing full matrix multiplication is slow, your code calculates this one row at a time. For a single row i, the error is $h^{(i)} - y^{(i)}$ (a scalar). You multiply this scalar by the row vector $x^{(i)}$:

$$\text{grad} \mathrel{+}= (h^{(i)} - y^{(i)}) \begin{bmatrix} 1 \\ x_{i,1} \\ x_{i,2} \end{bmatrix}$$

- **Code:** `grad[j] += error * X[i][j];`
- **Result:** `grad` is a 3×1 vector accumulating the total slope.

Step 4: Accumulating the Hessian (H)

The Hessian tells you the *curvature* of the loss function, which allows Newton's Method to take massive steps. The math formula for the whole dataset is:

$$H = \frac{1}{m} X^T S X$$

- Dimensions: $(3 \times m) \times (m \times m) \times (m \times 3) \to 3 \times 3$ matrix.

To avoid creating the $m \times m$ diagonal matrix S, your code calculates the **outer product** of the row $x^{(i)}$ with itself, weighted by the variance $h^{(i)}(1 - h^{(i)})$.

An outer product of a 3×1 vector and a 1×3 vector creates a 3×3 matrix:

$$H \mathrel{+}= h^{(i)}(1 - h^{(i)}) \begin{bmatrix} 1 \\ x_{i,1} \\ x_{i,2} \end{bmatrix} \begin{bmatrix} 1 & x_{i,1} & x_{i,2} \end{bmatrix}$$

$$H \mathrel{+}= h^{(i)}(1 - h^{(i)}) \begin{bmatrix} 1 \cdot 1 & 1 \cdot x_{i,1} & 1 \cdot x_{i,2} \\ x_{i,1} \cdot 1 & x_{i,1} \cdot x_{i,1} & x_{i,1} \cdot x_{i,2} \\ x_{i,2} \cdot 1 & x_{i,2} \cdot x_{i,1} & x_{i,2} \cdot x_{i,2} \end{bmatrix}$$

- **Code:** `H[j][l] += h * (1.0f - h) * X[i][j] * X[i][l];`
- **Result:** H is a 3×3 matrix accumulating the curvature information.

(After the `i` loop finishes, you divide both `grad` and `H` by m to get the average).

Step 5: The Matrix Inversion and Update (Newton's Step)

Now you have a 3×1 Gradient and a 3×3 Hessian.

The update rule for Newton's method is:

$$\theta_{\text{new}} = \theta_{\text{old}} - H^{-1}\nabla J$$

1. **Invert the Matrix:** You pass H into your `inversematrix` function.
   - **Result:** `H_inv` is a 3×3 matrix (H^{-1}).
2. **Multiply $H^{-1} \times \nabla J$:** You multiply a (3×3) matrix by a (3×1) vector, which results in a (3×1) update vector.
   - **Code:** `update_val += H_inv[j][l] * grad[l];`
3. **Update Weights:** You subtract this update vector from your current θ vector.
   - **Code:** `theta[j] -= update_val;`

This process repeats for `epoch` times. Because the Hessian accounts for the curvature of the loss space, it points almost exactly to the optimal solution immediately, meaning it only needs ~10 epochs compared to the 10,000 needed by Gradient Descent!